

ARCHITECTURAL DESIGN PATTERN (MVT)

Django follows the **MVT (Model-View-Template)** architectural pattern, which is a variation of the traditional MVC (Model-View-Controller) design pattern used in web development.

This pattern separates the application into three main components:

1. Model

Model acts as the data layer of an application. It defines the structure of the database and handles data-related logic.

- Models typically represent database tables.
- Responsible for querying, inserting, updating, and deleting data.
- Usually backed by relational databases such as MySQL, PostgreSQL, or SQLite.

2. View

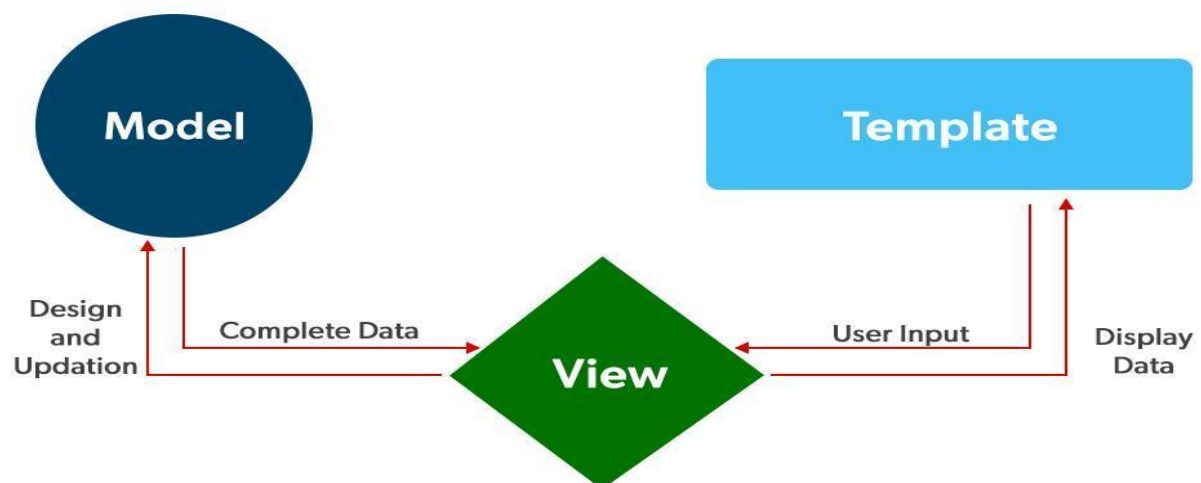
View handles business logic and user interface rendering. It processes user requests and prepares data for display.

- Interacts with models to fetch or update data.
- Passes data to templates for rendering.
- In Django, views are implemented as Python functions or classes that return HTTP responses.

3. Template

Template is responsible for presenting data to the user. It combines static content with dynamic data using template syntax.

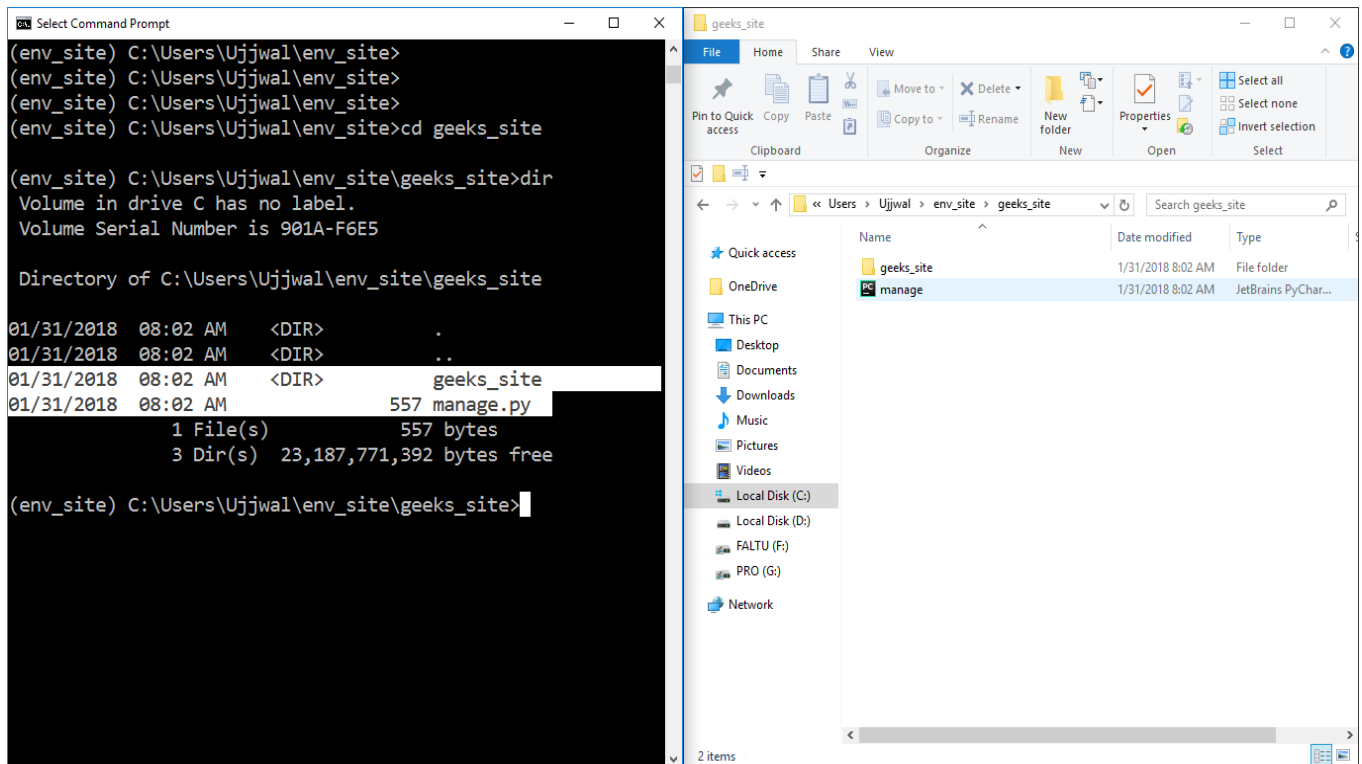
- Contains HTML, CSS, and JavaScript for the user interface.
- Uses Django Template Language (DTL) to insert dynamic data.
- Displays data passed from views in a structured format.



Project Structure

A Django Project when initialized contains basic files by default such as `manage.py`, `view.py`, etc. A simple project structure is enough to create a single-page application.

Here are the major files and their explanations. Inside the **geeks_site** folder (project folder) there will be the following files:

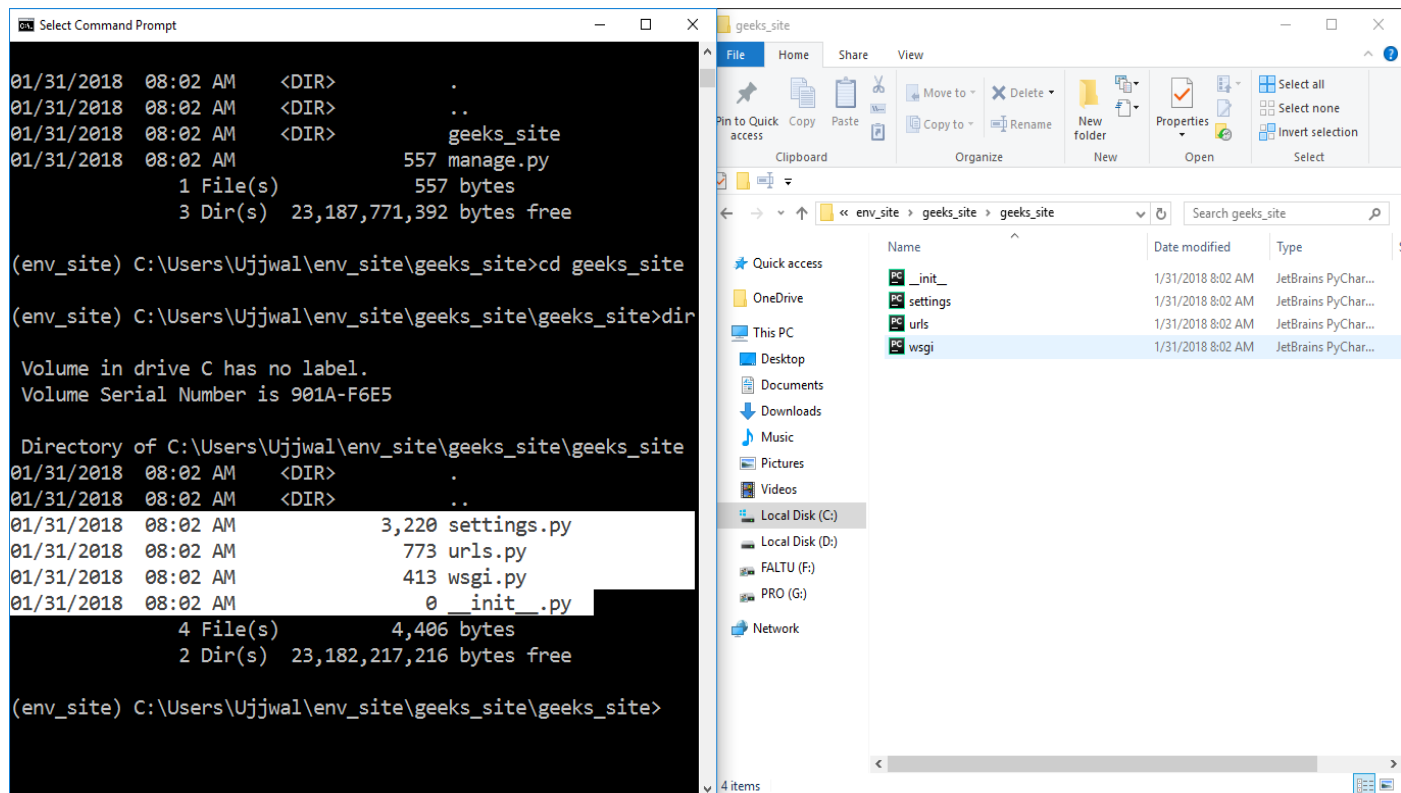


Explanation of Key Files and Folders

1. `manage.py`: This file is used to interact with your project via the command line(start the server, sync the database... etc). For getting the full list of commands that can be executed by `manage.py` type this code in the command window-

`$ python manage.py help`

2. folder (`geeks_site`): This folder contains all the packages of your project. Initially, it contains four files –



- **_init_.py**: Marks a directory as a Python package and can execute package-level initialization code.
- **settings.py**: Contains website settings, including app registrations, static file locations, and database configurations.
- **urls.py**: Stores all project URLs and maps them to corresponding view functions.
- **wsgi.py**: Used for deploying the project with WSGI and helps the Django application communicate with the web server.